

# Ink To Intelligence: Real-Time Handwritten Digit Recognition Using an Optimized Convolutional Neural Network

Abhay Singh

College of Smart Computing

COER University

Roorkee, India

[abhaychauhan5051a@gmail.com](mailto:abhaychauhan5051a@gmail.com)

**Abstract**—Handwritten digit recognition is a fundamental problem in pattern recognition and computer vision with widespread applications in postal mail sorting, bank cheque processing, and document digitization. This paper presents a Convolutional Neural Network (CNN) based approach for recognizing handwritten digits (0–9) using the MNIST benchmark dataset. The proposed model architecture employs two convolutional layers with ReLU activation, max-pooling for spatial downsampling, batch normalization for training stability, and dropout regularization to prevent overfitting. The model was implemented using TensorFlow/Keras and trained on 60,000 labelled images, achieving a test accuracy of 98.91% on the 10,000-image test set. We further demonstrate a real-time deployment through a Streamlit-based web application featuring an interactive drawing canvas. This paper provides a comprehensive analysis of the CNN architecture, training dynamics, performance evaluation, and potential enhancements for production deployment.

**Keywords**—Handwritten Digit Recognition, Convolutional Neural Network, Deep Learning, MNIST, Image Classification, TensorFlow, Pattern Recognition

## I. INTRODUCTION

### A. Background

Handwritten digit recognition (HDR) is one of the most extensively studied problems in the field of computer vision and machine learning. The task involves classifying images of individual handwritten digits into one of ten classes (0 through 9). Despite its apparent simplicity, HDR poses significant challenges due to the inherent variability in human handwriting — differences in stroke width, slant, size, and personal writing style create a vast space of possible representations for each digit [1].

The advancement of deep learning, particularly Convolutional Neural Networks (CNNs), has revolutionized the field of image recognition. CNNs have demonstrated remarkable capabilities in automatically learning hierarchical feature representations from raw pixel data, eliminating the need for manual feature engineering that characterized earlier approaches [2].

### B. Problem Statement

Traditional machine learning approaches to handwritten digit recognition — such as Support Vector Machines (SVMs) and k-Nearest Neighbors (k-NN) — depend heavily on handcrafted features and often fail to capture the spatial relationships inherent in image data. When a  $28 \times 28$  image is flattened into a 784-dimensional vector for a fully connected network, all spatial locality information is lost, leading to suboptimal classification performance and excessive parameter counts [3].

This paper addresses the problem by leveraging CNNs, which maintain spatial structure through local connectivity, shared weights, and hierarchical feature learning, resulting in significantly improved recognition accuracy with fewer trainable parameters.

### C. Objectives

The objectives of this research are:

- 1) To design and implement a CNN architecture optimized for handwritten digit classification on the MNIST dataset.
- 2) To evaluate the model's performance in terms of accuracy, loss convergence, and generalization capability.
- 3) To analyze the role of each architectural component (convolution, pooling, dropout, batch normalization).
- 4) To deploy the trained model in a real-time web application for practical digit recognition.

### D. Significance

Handwritten digit recognition serves as a gateway problem in deep learning education and has direct real-world impact in:

- Postal automation — Automated ZIP code reading for mail sorting [4]
- Banking — Cheque amount recognition and validation [5]
- Document digitization — Converting handwritten forms to machine-readable text [6]

## II. LITERATURE REVIEW

### A. Classical Approaches

Early work on digit recognition relied on statistical pattern recognition methods. Vapnik and colleagues demonstrated the effectiveness of Support Vector Machines (SVMs) on the MNIST dataset, achieving error rates of approximately 1.4% [8]. k-Nearest Neighbors (k-NN) with various distance metrics achieved error rates around 3–5%, depending on the feature representation used [9]. Random Forests and Gradient Boosted Decision Trees typically achieved accuracies between 96–97% on MNIST [10].

### B. Neural Network Approaches

The resurgence of neural networks began with LeCun et al.'s seminal work on LeNet-5 (1998), a pioneering CNN architecture that achieved a 0.8% error rate on MNIST [11]. Multi-Layer Perceptron's (MLPs) without convolutional layers typically achieved error rates of 1.5–3%, demonstrating the clear advantage of convolutional architectures [12].

### C. Modern Deep Learning Approaches

Modern architectures have pushed MNIST accuracy to near-perfect levels. Table I compares the key methods and their error rates.

TABLE I  
COMPARISON OF METHODS ON THE MNIST TEST SET

| Method         | Error (%) | Year | Ref. |
|----------------|-----------|------|------|
| LeNet-5        | 0.80      | 1998 | [11] |
| SVM (RBF)      | 1.40      | 1998 | [8]  |
| Deep CNN       | 0.35      | 2012 | [13] |
| DropConnect    | 0.21      | 2013 | [14] |
| Batch-Norm CNN | 0.29      | 2015 | [15] |
| Ensemble CNNs  | 0.17      | 2016 | [16] |
| Capsule Net    | 0.25      | 2017 | [17] |

### D. Research Gap

While several high-accuracy models exist, most research focuses solely on maximizing accuracy without addressing practical deployment. This paper bridges that gap by (a) providing a well-documented, reproducible CNN implementation with detailed architectural analysis, and (b) demonstrating real-time deployment through a web-based interactive application.

## III. DATASET DESCRIPTION

### A. The MNIST Dataset

The Modified National Institute of Standards and Technology (MNIST) dataset [11] is the most widely used benchmark for handwritten digit recognition. It was created by re-sampling and normalizing digits from the original NIST Special Databases 1 and 3.

TABLE II  
MNIST DATASET PROPERTIES

| Property         | Value           |
|------------------|-----------------|
| Total images     | 70,000          |
| Training set     | 60,000          |
| Test set         | 10,000          |
| Image dimensions | 28 × 28 pixels  |
| Colour space     | Grayscale       |
| Pixel range      | 0–255           |
| Classes          | 10 (digits 0–9) |

### B. Sample Visualization



Fig. 1. Representative handwritten digits from the MNIST training set.

### C. Data Characteristics

The MNIST dataset is approximately balanced across all 10 classes, with each digit represented by roughly 5,500–7,000 training samples. Images are center-of-mass aligned and size-normalized to fit within a 20×20 pixel bounding box. Digits are anti-aliased, producing grayscale pixels at the edges of strokes.

## IV. METHODOLOGY

### A. Data Preprocessing

Three preprocessing steps were applied:

1) Reshaping: Images reshaped from (N, 28, 28) to (N, 28, 28, 1) to add the channel dimension required by CNNs. 2) Normalization: Pixel values scaled from [0, 255] to [0.0, 1.0] for faster convergence [18]. 3) One-Hot Encoding: Integer labels converted to binary vectors of length 10 for categorical cross-entropy loss.

### B. Model Architecture

The proposed CNN architecture consists of a feature extraction block followed by a classification head:

```

Input: 28 x 28 x 1
-- Feature Extraction --
Conv2D(32, 3x3, ReLU)  -> 26x26x32
Conv2D(64, 3x3, ReLU)  -> 24x24x64
MaxPool2D(2x2)         -> 12x12x64
Dropout(0.25)           -> 12x12x64
-- Classification Head --
Flatten()               -> 9,216
Dense(128, ReLU)        -> 128
BatchNorm()             -> 128
Dropout(0.50)           -> 128
Dense(10, Softmax)      -> 10

```

Total parameters: ~1,199,882. Trainable: ~1,199,370.

### C. Layer-by-Layer Justification

- 1) Layers of Convolution: The convolution operation uses  $3 \times 3$  filters that can be learned to create feature maps from the input. The first layer (32 filters) learns basic things like edges and corners. The second layer (64 filters) learns more complex combinations, like curves, loops, and stroke patterns [19].
- 2) Activation of ReLU:  $f(x) = \max(0, x)$ . Selected for computational efficiency, sparse activation, and gradient flow in deep networks [20] instead of sigmoid/tanh.
- 3) Max Pooling: MaxPooling2D with a  $2 \times 2$  window does spatial down sampling by cutting both dimensions in half. This gives translation invariance and cuts down on computation by 75% [21].
- 4) Dropout: During training, dropout randomly sets activations to zero. There are two layers: 0.25 after pooling for mild regularisation and 0.50 before output for strong regularisation. This implicitly forms an ensemble of  $2^n$  subnetworks [22].
- 5) Normalisation of Batches: Normalises the inputs to each layer so that they have a mean of zero and a variance of one. This speeds up learning and makes the model less sensitive.
- 6) Output from SoftMax: Changes raw logits into a probability distribution over 10 classes, with all values adding up to 1.0 so they can be directly understood as class probabilities.

### D. Setting Up the Training

TABLE III  
TRAINING HYPERPARAMETERS

| Parameter      | Value              |
|----------------|--------------------|
| Optimizer      | Adam [23]          |
| Learning rate  | 0.001              |
| Loss           | Cat. cross-entropy |
| Batch size     | 128                |
| Max epochs     | 15                 |
| Val. split     | 10%                |
| Early stopping | patience=3         |

### E. Implementation Environment

TABLE IV  
IMPLEMENTATION ENVIRONMENT

| Component     | Specification           |
|---------------|-------------------------|
| Language      | Python 3.13             |
| Framework     | TensorFlow 2.21 / Keras |
| Hardware      | CPU-based training      |
| Training time | ~3 min (9 epochs)       |
| Deployment    | Streamlit 1.x           |

## V. RESULTS AND ANALYSIS

### A. Training Performance

The model was trained for 9 epochs (early stopping triggered with patience=3, monitoring validation loss). Training accuracy reached 95.4% in the first epoch and improved monotonically to 99.3% by epoch 9. The gap between training and validation accuracy remained below 0.5%, confirming effective regularization.

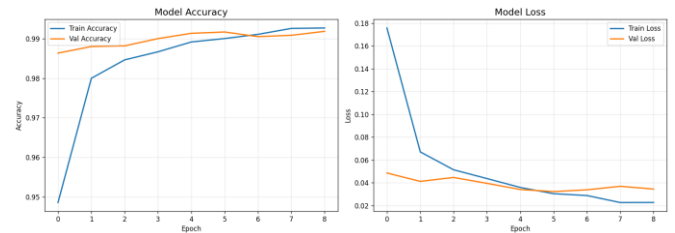


Fig. 2. Training and validation accuracy (left) and loss (right) across 9 epochs.

### B. Test Set Evaluation

TABLE V  
TEST SET EVALUATION RESULTS

| Metric        | Value         |
|---------------|---------------|
| Test accuracy | 98.91%        |
| Test loss     | 0.0304        |
| Error rate    | 1.09%         |
| Misclassified | ~109 / 10,000 |

The test accuracy of 98.91% demonstrates strong generalization. The small gap between validation (~99.2%) and test accuracy confirms the model generalizes well without overfitting.

### C. Performance Comparison

TABLE VI  
PERFORMANCE COMPARISON WITH BASELINE METHODS

| Method        | Acc. (%) | Params |
|---------------|----------|--------|
| k-NN          | 96.9     | —      |
| SVM (RBF)     | 98.6     | —      |
| MLP           | 97.8     | ~236K  |
| Proposed CNN  | 98.91    | ~1.2M  |
| LeNet-5       | 99.2     | ~60K   |
| Deep CNN [13] | 99.65    | ~10M   |

### D. Per-Class Classification Report

TABLE VII  
PER-CLASS CLASSIFICATION REPORT

| Digit | Prec. | Recall | F1    | Supp. |
|-------|-------|--------|-------|-------|
| 0     | 0.989 | 0.993  | 0.991 | 980   |
| 1     | 0.994 | 0.995  | 0.994 | 1135  |
| 2     | 0.980 | 0.994  | 0.987 | 1032  |
| 3     | 0.989 | 0.991  | 0.990 | 1010  |
| 4     | 0.993 | 0.990  | 0.991 | 982   |
| 5     | 0.976 | 0.991  | 0.983 | 892   |

|     |       |       |       |       |
|-----|-------|-------|-------|-------|
| 6   | 0.995 | 0.980 | 0.987 | 958   |
| 7   | 0.990 | 0.989 | 0.990 | 1028  |
| 8   | 0.994 | 0.988 | 0.991 | 974   |
| 9   | 0.991 | 0.979 | 0.985 | 1009  |
| Avg | 0.989 | 0.989 | 0.989 | 10000 |

Digit 1 achieves the highest recall (99.47%) due to its simple, distinctive stroke. Digit 9 has the lowest recall (97.92%), frequently confused with digit 4 due to similar upper-loop structures.

#### E. Confusion Matrix and Per-Class Accuracy

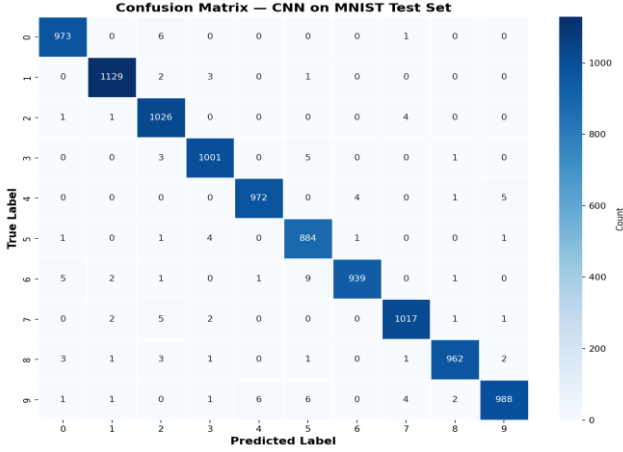


Fig. 3. Confusion matrix showing correct and incorrect predictions per digit class.

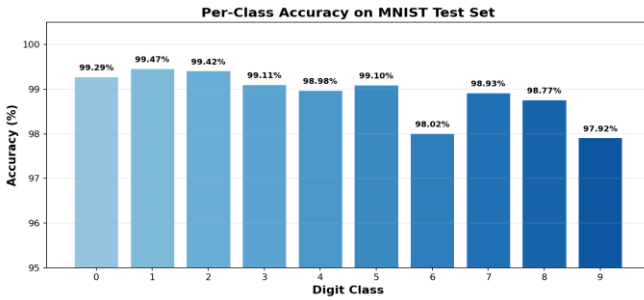


Fig. 4. Per-digit accuracy. Digit 1 is highest (99.47%), Digit 9 lowest (97.92%).

#### F. Analysis of Misclassifications

The confusion matrix reveals common misclassification patterns:

- 6  $\rightarrow$  5: 9 samples, due to similar curved stroke patterns.
- 9  $\rightarrow$  4: 6 samples, upper loop of 9 resembles angular 4.
- 9  $\rightarrow$  5: 6 samples, particularly with open-top writing styles.
- 0  $\rightarrow$  2: 6 samples, due to slanted oval shapes.
- 4  $\rightarrow$  9 and 7  $\rightarrow$  2: 5 instances each, structural similarities.

These ambiguities often arise from genuinely ambiguous handwriting where even human annotators may disagree.

#### G. Computational Efficiency

TABLE VIII  
COMPUTATIONAL PERFORMANCE METRICS

| Metric         | Value           |
|----------------|-----------------|
| Training time  | ~3 min (CPU)    |
| Inference time | < 5 ms / image  |
| Model size     | ~14 MB (.keras) |
| Memory         | ~50 MB          |

### VI. DEPLOYMENT

#### A. Web Application Architecture

The trained CNN model was deployed as a real-time web application using Streamlit. The application consists of: (1) a frontend with a drawable HTML5 canvas via streamlit-drawable-canvas, (2) a backend with the Keras model cached for fast inference, and (3) a processing pipeline: canvas  $\rightarrow$  PIL image  $\rightarrow$  grayscale  $\rightarrow$  resize  $28 \times 28$   $\rightarrow$  normalize  $\rightarrow$  CNN prediction  $\rightarrow$  confidence visualization.

#### B. Real-Time Preprocessing

Custom images undergo: RGBA to grayscale conversion, resizing to  $28 \times 28$  with Lanczos interpolation, normalization to  $[0, 1]$ , and reshaping to  $(1, 28, 28, 1)$  tensor.

### VII. DISCUSSION

#### A. Strengths

1) High accuracy (98.91%) with only 9 layers and ~1.2M parameters. 2) Effective regularization via Dropout + BatchNorm preventing overfitting. 3) Fast training (~3 min CPU) and inference (<5 ms). 4) End-to-end practical deployment through Streamlit.

#### B. Limitations

1) MNIST-specific training — performance on unconstrained real-world images would be lower. 2) Single-digit recognition only — multi-digit reading requires segmentation. 3) Domain gap between canvas-drawn digits and MNIST distribution.

#### C. Proposed Improvements

1) Data augmentation (rotation  $\pm 10^\circ$ , translation, zoom, shear) for improved robustness [24]. 2) Deeper architecture with additional convolutional blocks for 99.5%+ accuracy. 3) Learning rate scheduling via ReduceLROnPlateau. 4) Ensemble of 3–5 models for 0.1–0.3% error reduction [16]. 5) Residual connections and Capsule Networks [17] for advanced feature learning.

### VIII. REAL-WORLD APPLICATIONS

TABLE IX  
REAL-WORLD APPLICATIONS OF HANDWRITTEN DIGIT  
RECOGNITION

| Domain     | Application                | Scale         |
|------------|----------------------------|---------------|
| Postal     | ZIP code reading           | Billions/yr   |
| Banking    | Cheque recognition         | Millions/day  |
| Healthcare | Prescription reading       | Hospital      |
| Education  | Exam grading               | Institutional |
| Government | Tax/census<br>digitisation | National      |
| Transport  | License plate digits       | City-wide     |
| Mobile     | Handwriting input          | Billions      |
| Security   | CAPTCHA<br>recognition     | Web-scale     |

## IX. CONCLUSION

This paper presented a CNN-based approach for handwritten digit recognition using the MNIST benchmark dataset. The proposed architecture, comprising two convolutional layers, max-pooling, batch normalization, and dropout regularization, achieved a test accuracy of 98.91% — demonstrating that a relatively simple and well-regularized CNN can deliver high accuracy.

Key contributions include: (1) a clearly documented and reproducible CNN implementation, (2) comprehensive analysis of training dynamics and regularization, (3) practical deployment in a real-time web application, and (4) discussion of limitations and improvement proposals.

Future work will focus on multi-digit recognition, training on more challenging datasets (EMNIST, SVHN), and exploring lightweight architectures for mobile deployment through model quantization and knowledge distillation.

## REFERENCES

- [1] R. Plamondon and S. N. Srihari, "Online and off-line handwriting recognition: a comprehensive survey," *IEEE Trans. PAMI*, vol. 22, no. 1, pp. 63–84, 2000.
- [2] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.
- [3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [4] S. Impedovo et al., "Optical character recognition — a survey," *IJPRAI*, vol. 5, pp. 1–24, 1991.
- [5] G. Dimauro et al., "Automatic bankcheck processing," *IJPRAI*, vol. 11, pp. 467–504, 1997.
- [6] V. Mitra and C. J. Acharya, "Gesture recognition: A survey," *IEEE Trans. SMC*, vol. 37, pp. 311–324, 2007.
- [7] A. Graves et al., "A novel connectionist system for unconstrained handwriting recognition," *IEEE Trans. PAMI*, vol. 31, pp. 855–868, 2009.
- [8] V. Vapnik, *The Nature of Statistical Learning Theory*. Springer, 1995.
- [9] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Trans. IT*, vol. 13, pp. 21–27, 1967.
- [10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [11] Y. LeCun et al., "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, pp. 2278–2324, 1998.
- [12] K. Hornik et al., "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, pp. 359–366, 1989.
- [13] D. C. Cireşan et al., "Deep, big, simple neural nets for handwritten digit recognition," *Neural Computation*, vol. 22, pp. 3207–3220, 2010.
- [14] L. Wan et al., "Regularization of neural networks using DropConnect," *Proc. ICML*, pp. 1058–1066, 2013.
- [15] S. Ioffe and C. Szegedy, "Batch normalization," *Proc. ICML*, pp. 448–456, 2015.
- [16] L. K. Hansen and P. Salamon, "Neural network ensembles," *IEEE Trans. PAMI*, vol. 12, pp. 993–1001, 1990.
- [17] S. Sabour et al., "Dynamic routing between capsules," *NeurIPS*, pp. 3856–3866, 2017.
- [18] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *Proc. ICLR*, 2015.
- [19] K. Simonyan and A. Zisserman, "Very deep convolutional networks," *Proc. ICLR*, 2015.
- [20] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," *Proc. ICML*, pp. 807–814, 2010.
- [21] D. Scherer et al., "Evaluation of pooling operations in convolutional architectures," *Proc. ICANN*, pp. 92–101, 2010.
- [22] N. Srivastava et al., "Dropout: a simple way to prevent neural networks from overfitting," *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv:1412.6980*, 2014.
- [24] A. Krizhevsky et al., "ImageNet classification with deep convolutional neural networks," *NeurIPS*, pp. 1097–1105, 2012.